

03 — Template OpenShift e Helm Charts

Modulo: 3 — Deploy di Applicazioni

Versione: OpenShift 4.14+ su Azure ARO

Obiettivi di Apprendimento

- Comprendere la struttura di un Template OpenShift e il suo sistema di parametri
 - Istanziare un Template con `oc process` e `oc apply` passando parametri personalizzati
 - Installare e usare Helm CLI su OpenShift per il deploy di applicazioni
 - Distinguere Template OCP e Helm Chart scegliendo lo strumento appropriato per ogni scenario
 - Navigare il catalogo di Template e Helm Charts dalla Console Web ARO
 - Applicare considerazioni di sicurezza (SCC) quando si installano Helm Charts su ARO
-

1. Teoria e Concetti Chiave

Template OpenShift

Un **Template** OpenShift è un oggetto Kubernetes personalizzato (`Kind: Template`) che raggruppa un insieme di risorse (Deployment, Service, Route, ConfigMap, ecc.) con **parametri** sostituibili al momento dell'istanziamento.

Vantaggi dei Template:

- Un singolo Template può essere istanziato più volte con valori diversi (es. ambiente dev/staging/prod)
- I parametri possono avere valori di default e generazione automatica (es. password casuali)
- Integrati nella Console Web di OCP nel catalogo di servizi

Struttura di un Template:

```
Template
├── metadata      → nome, namespace, descrizione, tag per il catalogo
├── parameters    → lista di parametri con nome, descrizione, default,
generazione
├── objects       → lista delle risorse da creare (Deployment, Service, Route,
ecc.)
└── labels        → label applicate a tutte le risorse create
```

Helm su OpenShift

Helm è il package manager de facto per Kubernetes. Un **Helm Chart** è un pacchetto che contiene:

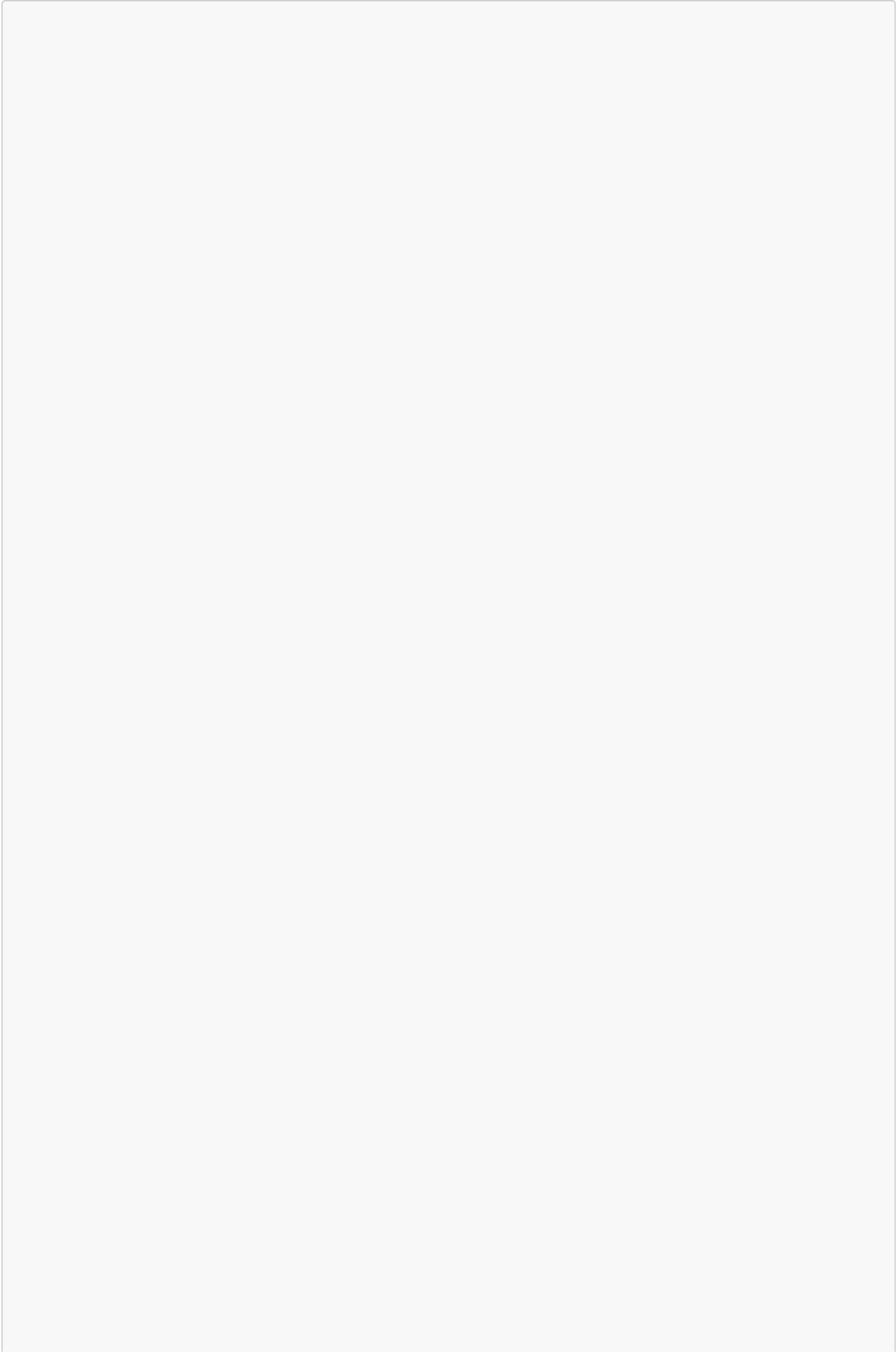
- `Chart.yaml` — metadati del chart (nome, versione, descrizione)
- `values.yaml` — valori di default configurabili
- `templates/` — template Go delle risorse Kubernetes/OCP

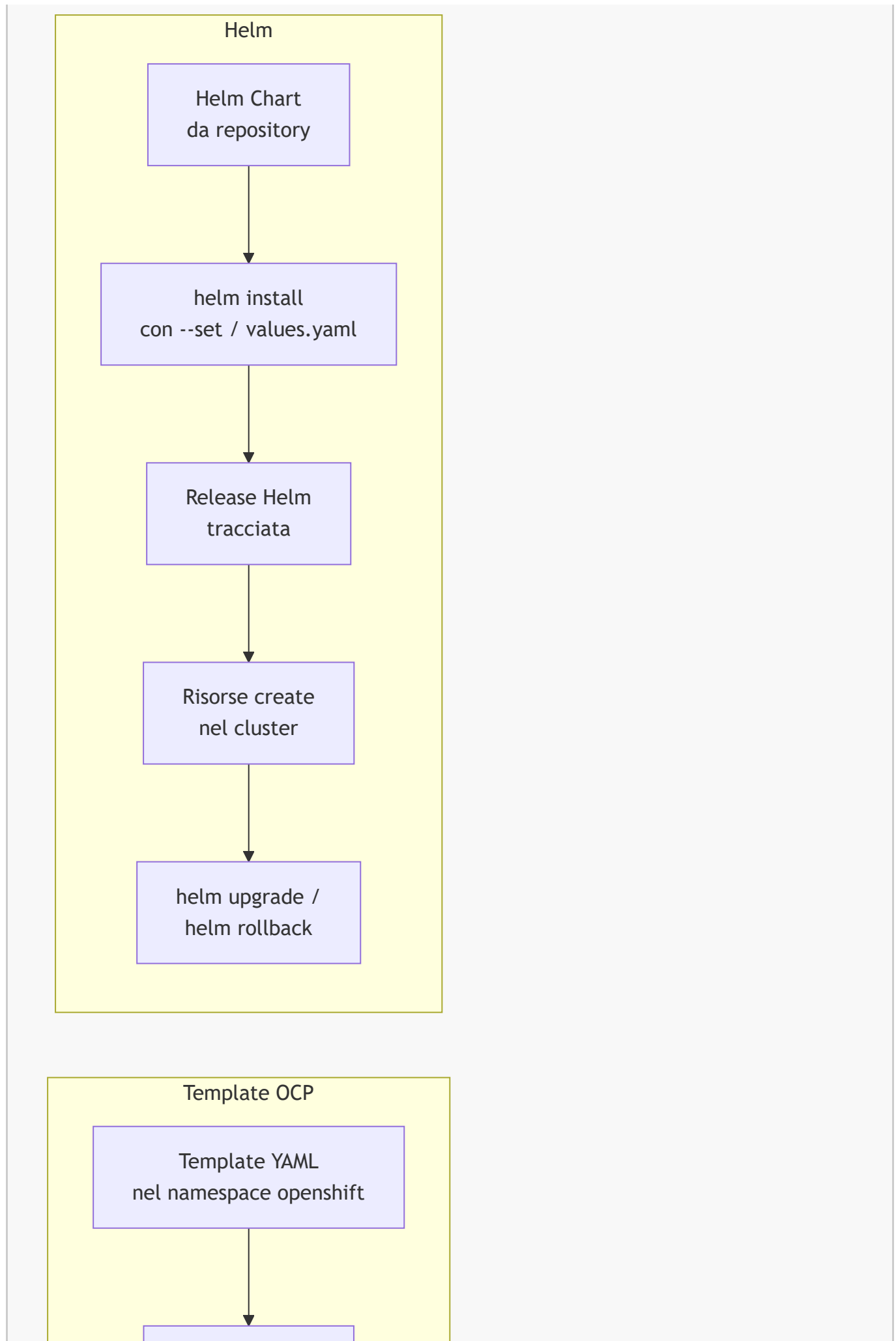
- [charts/](#) — dipendenze da altri chart

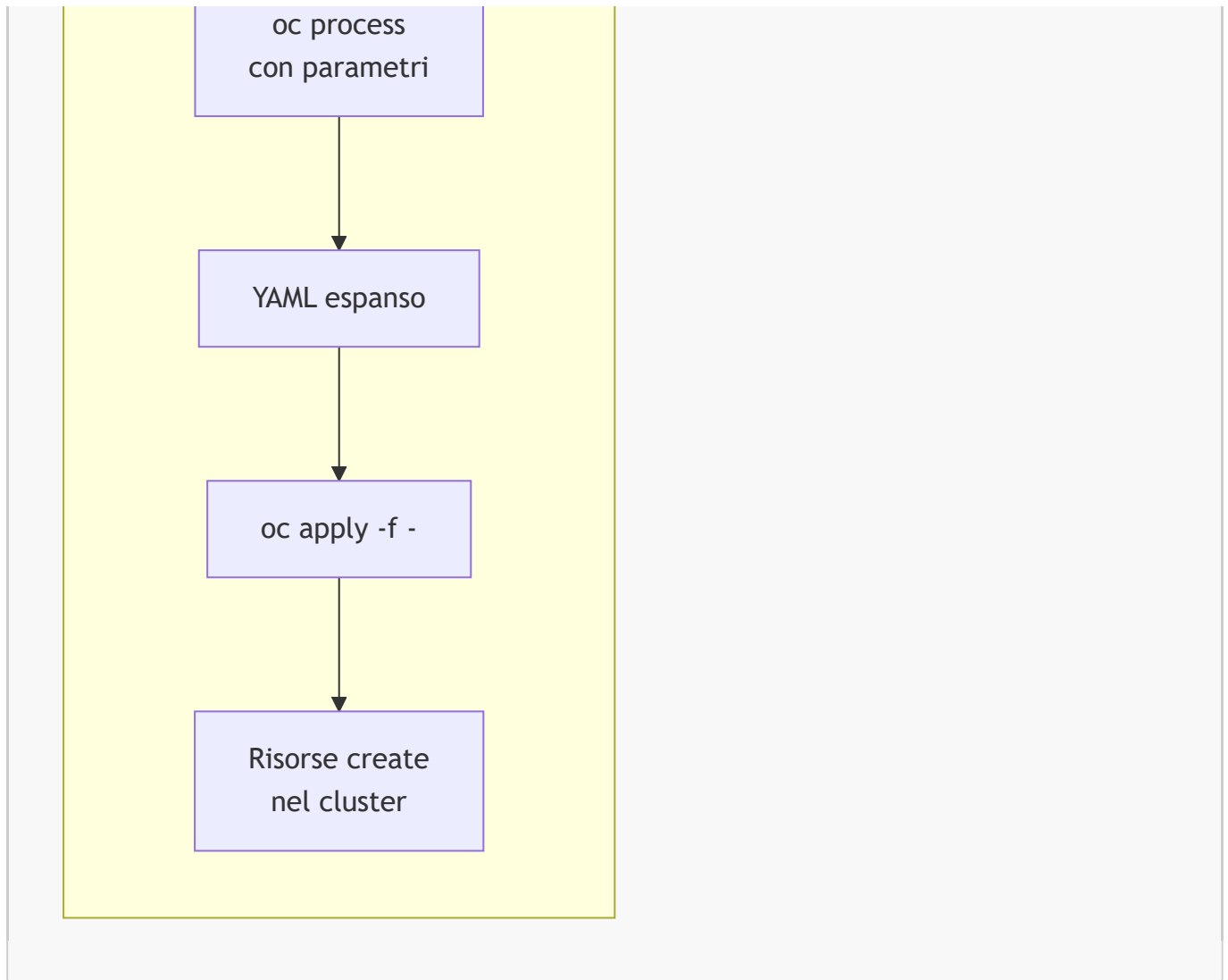
Template OCP vs Helm Chart

Caratteristica	Template OCP	Helm Chart
Sintassi parametri	<code>\${NOME_PARAMETRO}</code>	<code>{{ .Values.nome }}</code> (Go template)
Gestione release	Nessuna (apply una volta)	Helm traccia release e versioni
Rollback	Manuale	<code>helm rollback</code> automatico
Dipendenze	Non supportate	<code>Chart.yaml dependencies</code>
Ecosistema	Solo OCP/RedHat	Universale Kubernetes
Curva apprendimento	Bassa	Media
Quando usarlo	Template semplici, catalogo OCP	Applicazioni complesse, multi-ambiente

2. Diagramma — Flusso Template vs Helm







3. Comandi Pratici con Output Atteso

```

# Visualizzare tutti i Template disponibili nel namespace 'openshift' (catalogo
predefinito)
oc get templates -n openshift

# Output atteso (estratto):
# NAME                                DESCRIPTION
PARAMETERS    OBJECTS
# cakephp-mysql-persistent            An example CakePHP application with a MySQL...
20 (4 blank)   8
# dancer-mysql-persistent            An example Dancer application with a MySQL...
17 (5 blank)   8
# django-psql-persistent            An example Django application with a PostgreSQL...
18 (5 blank)   8
# nodejs-postgresql-persistent      An example Node.js application with PostgreSQL...
18 (5 blank)   8
# rails-postgresql-persistent        An example Rails application with PostgreSQL...
20 (4 blank)   8

# Descrivere un Template per vedere tutti i parametri disponibili
oc describe template nodejs-postgresql-persistent -n openshift

```

```
# Elencare solo i parametri di un Template
oc process --parameters nodejs-postgresql-persistent -n openshift

# Output atteso:
# NAME                                DESCRIPTION                                GENERATOR  VALUE
# NAME                                The name assigned to all...                nodejs-app
# NAMESPACE                           The OpenShift Namespace...                my-project
# MEMORY_LIMIT                         Maximum amount of memory...                512Mi
# SOURCE_REPOSITORY_URL                The URL of the repository...
https://github.com/...
# DATABASE_SERVICE_NAME                The name of the OpenShift...                postgresql
# POSTGRESQL_USER                      Username for PostgreSQL...                expression user[A-Z0-9]
{3}
# POSTGRESQL_PASSWORD                 Password for the user...                expression [a-zA-Z0-9]
{16}

# Istanziare il Template con valori personalizzati
oc process nodejs-postgresql-persistent \
  -n openshift \
  -p NAME=my-nodejs-app \
  -p SOURCE_REPOSITORY_URL=https://github.com/myorg/myapp.git \
  -p MEMORY_LIMIT=256Mi | oc apply -f -

# Output atteso:
# imagestream.image.openshift.io/my-nodejs-app created
# buildconfig.build.openshift.io/my-nodejs-app created
# deploymentconfig.apps.openshift.io/my-nodejs-app created
# service/my-nodejs-app created
# route.route.openshift.io/my-nodejs-app created
# secret/my-nodejs-app created
# service/postgresql created
# deploymentconfig.apps.openshift.io/postgresql created

# Salvare l'output del processo su file (per GitOps)
oc process nodejs-postgresql-persistent \
  -n openshift \
  -p NAME=my-nodejs-app \
  -o yaml > my-nodejs-app-manifests.yaml

# --- Helm ---

# Verificare che Helm sia installato
helm version

# Output atteso:
# version.BuildInfo{Version:"v3.14.0", GitCommit:"...", GoVersion:"go1.21.5"}

# Aggiungere il repository Bitnami
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Output atteso:
# "bitnami" has been added to your repositories
```

```
# Hang tight while we grab the latest from your chart repositories...
# ...Successfully got an update from the "bitnami" chart repository
# Update Complete. *Happy Helming!*

# Cercare un chart nel repository
helm search repo bitnami/nginx

# Output atteso:
# NAME          CHART VERSION  APP VERSION DESCRIPTION
# bitnami/nginx 15.14.0        1.25.3        NGINX Open Source is a web server...

# Installare un chart Helm nel namespace corrente
helm install my-nginx bitnami/nginx \
  --namespace my-project \
  --set service.type=ClusterIP \
  --set replicaCount=2

# Output atteso:
# NAME: my-nginx
# LAST DEPLOYED: Sun May 24 10:30:00 2026
# NAMESPACE: my-project
# STATUS: deployed
# REVISION: 1

# Elencare le release Helm nel namespace
helm list -n my-project

# Output atteso:
# NAME          NAMESPACE    REVISION    UPDATED           STATUS    CHART
# APP VERSION
# my-nginx      my-project    1           2026-05-24...    deployed  nginx-15.14.0
# 1.25.3

# Aggiornare una release Helm
helm upgrade my-nginx bitnami/nginx \
  --namespace my-project \
  --set replicaCount=3

# Eseguire il rollback alla revisione precedente
helm rollback my-nginx 1 -n my-project

# Disinstallare una release Helm
helm uninstall my-nginx -n my-project

# Visualizzare i valori di default di un chart
helm show values bitnami/nginx

# Installare con file values personalizzato
helm install my-nginx bitnami/nginx \
  -f my-values.yaml \
  -n my-project
```

4. Esempi YAML Completi

Template OpenShift personalizzato

```
# file: template-webapp.yaml
apiVersion: template.openshift.io/v1
kind: Template
metadata:
  name: webapp-template
  namespace: my-project
  annotations:
    description: "Template per una semplice applicazione web con Service e Route"
    iconClass: icon-nodejs
    tags: "nodejs,webapp" # tag visibili nel catalogo OCP
  labels:
    template: webapp-template
# Parametri sostituibili
parameters:
- name: APP_NAME
  description: "Nome dell'applicazione (usato per tutte le risorse)"
  required: true
  value: "my-webapp"
- name: APP_IMAGE
  description: "Immagine container da usare"
  required: true
  value: "registry.access.redhat.com/ubi9/ubi-minimal:latest"
- name: REPLICAS
  description: "Numero di repliche iniziali"
  value: "2"
- name: MEMORY_LIMIT
  description: "Limite di memoria per container"
  value: "256Mi"
- name: APP_SECRET
  description: "Chiave segreta generata automaticamente"
  generate: expression # generato automaticamente
  from: "[a-zA-Z0-9]{32}"
# Risorse che il Template creerà
objects:
- apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: "${APP_NAME}" # ${PARAMETRO} viene sostituito al momento
    dell'istanziamento
    labels:
      app: "${APP_NAME}"
  spec:
    replicas: "${REPLICAS}" # ${} per valori numerici (no virgolette)
    selector:
      matchLabels:
        app: "${APP_NAME}"
    template:
      metadata:
```



```

    labels:
      app: "${APP_NAME}"
  spec:
    containers:
      - name: "${APP_NAME}"
        image: "${APP_IMAGE}"
        imagePullPolicy: Always
        ports:
          - containerPort: 8080
        resources:
          limits:
            memory: "${MEMORY_LIMIT}"
          requests:
            memory: "128Mi"
            cpu: "100m"
        env:
          - name: APP_SECRET
            value: "${APP_SECRET}"
- apiVersion: v1
  kind: Service
  metadata:
    name: "${APP_NAME}"
    labels:
      app: "${APP_NAME}"
  spec:
    selector:
      app: "${APP_NAME}"
    ports:
      - port: 80
        targetPort: 8080
- apiVersion: route.openshift.io/v1
  kind: Route
  metadata:
    name: "${APP_NAME}"
    labels:
      app: "${APP_NAME}"
  spec:
    to:
      kind: Service
      name: "${APP_NAME}"
    port:
      targetPort: 8080
    tls:
      termination: edge
      insecureEdgeTerminationPolicy: Redirect

```

```

# Installare il Template nel namespace openshift (visibile a tutti gli utenti)
oc apply -f template-webapp.yaml -n openshift

# Istanziare il Template
oc process webapp-template \

```

```
-n openshift \  
-p APP_NAME=my-webapp \  
-p REPLICAS=3 | oc apply -n my-project -f -
```

File `values.yaml` personalizzato per Helm

```
# file: my-nginx-values.yaml  
# Valori personalizzati per bitnami/nginx su OpenShift ARO  
  
replicaCount: 2  
  
image:  
  repository: quay.io/bitnami/nginx  
  tag: "1.25"  
  pullPolicy: IfNotPresent  
  
service:  
  type: ClusterIP      # su OCP si usa Route, non LoadBalancer  
  port: 80  
  
# Configurazione specifica OpenShift  
# bitnami/nginx su OCP richiede containerSecurityContext non-root  
containerSecurityContext:  
  enabled: true  
  runAsUser: 1001      # UID non-root richiesto da SCC restricted-v2  
  runAsNonRoot: true  
  allowPrivilegeEscalation: false  
  capabilities:  
    drop: ["ALL"]  
  
resources:  
  requests:  
    cpu: 100m  
    memory: 128Mi  
  limits:  
    cpu: 500m  
    memory: 256Mi
```

5. Note Specifiche per Azure ARO

⚠ Attenzione ARO — Helm e SCC: Molti Helm Chart pubblici (Bitnami, Artifact Hub) sono progettati per Kubernetes vanilla e includono `securityContext` che potrebbero essere incompatibili con il SCC `restricted-v2` di ARO. Prima di installare un chart, verificare il `values.yaml` e sovrascrivere i valori di sicurezza.

```
# Verificare se un chart Helm è compatibile con restricted-v2 prima  
dell'installazione
```

```

helm template my-nginx bitnami/nginx \
  --namespace my-project \
  --set service.type=ClusterIP | oc auth can-i create -f - --
as=system:serviceaccount:my-project:default

# Installare il repository Helm integrato in OCP (Red Hat Helm Charts)
helm repo add openshift-helm-charts https://charts.openshift.io/
helm repo update
helm search repo openshift-helm-charts

# Output atteso:
# NAME                                CHART VERSION  APP VERSION
DESCRIPTION
# openshift-helm-charts/redhat-keycloak 22.0.3         22.0.3        ...
# openshift-helm-charts/redhat-postgresql ...

# Su ARO, per chart che usano PersistentVolumeClaim, verificare la StorageClass
disponibile
oc get storageclass

# Output atteso (ARO):
# NAME                                PROVISIONER          RECLAIMPOLICY
VOLUMEBINDINGMODE
# managed-csi (default) disk.csi.azure.com    Delete
WaitForFirstConsumer
# managed-csi-premium  disk.csi.azure.com    Delete
WaitForFirstConsumer
# azurefile-csi        file.csi.azure.com    Delete          Immediate
# azurefile-csi-premium file.csi.azure.com    Delete          Immediate

# Specificare la StorageClass ARO durante l'installazione Helm
helm install my-postgresql bitnami/postgresql \
  --namespace my-project \
  --set global.storageClass=managed-csi-premium \
  --set primary.persistence.size=10Gi

```

Aspetto	OCP On-Premise	Azure ARO
Catalogo template	Gestito dall'admin	Pre-popolato da Red Hat
Helm charts	Qualsiasi	Preferire openshift-helm-charts per compatibilità
SCC compatibilità	restricted-v2	restricted-v2 (stesso, ma più enforcement)
StorageClass	Dipende dall'infrastruttura	managed-csi (Azure Disk) default
Route TLS	Wildcard gestito dall'admin	Wildcard ARO *.apps.<cluster>.aroapp.io

6. Riepilogo dei Punti Chiave

- Un **Template OCP** raggruppa più risorse con parametri sostituibili; **oc process** genera il YAML espanso e **oc apply** lo applica

- I parametri possono avere un generatore automatico (`generate: expression`) utile per password e chiavi segrete
 - **Helm** traccia le release con versioning, permettendo upgrade e rollback controllati: vantaggio chiave rispetto ai Template
 - Usare Template OCP per deploy semplici e integrazione con il catalogo; usare Helm per applicazioni complesse con dipendenze
 - Su **ARO** molti chart pubblici Bitnami richiedono override dei `securityContext` per essere compatibili con `restricted-v2`
 - Il repository `openshift-helm-charts` (Red Hat) fornisce chart già compatibili con OCP/ARO senza modifiche
 - Per PVC con Helm su ARO, specificare sempre `--set global.storageClass=managed-csi` (o `managed-csi-premium`)
-

7. Domande di Verifica

1. Quale comando genera il YAML espanso da un Template OpenShift senza applicarlo al cluster?

- A) `oc apply template/nodejs-app`
- B) `oc get template nodejs-app -o yaml`
- C) `oc process nodejs-app -n openshift -p NAME=myapp -o yaml` ✓
- D) `oc create template nodejs-app`

2. Qual è il vantaggio principale di Helm rispetto ai Template OpenShift?

- A) Helm supporta più linguaggi di programmazione
- B) Helm funziona solo su OpenShift
- C) **Helm traccia le release con versioning e supporta rollback automatico con `helm rollback`** ✓
- D) I Template OCP non supportano parametri

3. In un Template OCP, quale sintassi si usa per un parametro con valore numerico (es. numero di repliche)?

- A) `"${REPLICAS}"`
- B) `${REPLICAS}`
- C) `{{ .Values.replicas }}`
- D) `${{REPLICAS}}` ✓

4. Su ARO, quale problema comune si incontra installando Helm Charts da repository pubblici?

- A) Helm non è supportato su ARO
- B) I chart pubblici usano solo immagini Docker Hub non accessibili da ARO
- C) **Molti chart pubblici configurano `securityContext` incompatibili con il SCC `restricted-v2` di ARO** ✓
- D) I chart pubblici non supportano PersistentVolumeClaim

5. Quale repository Helm è consigliato su OCP/ARO per avere chart già compatibili con le restrizioni di sicurezza?

- A) `stable` (repository Helm ufficiale)
- B) `bitnami`

- C) `artifact-hub`
- D) `openshift-helm-charts` (`charts.openshift.io`) 